

Final Project Report

Introduction

This project presents a simplified aircraft maintenance management system designed to demonstrate modern containerization and deployment practices. In aviation, accurately tracking maintenance tasks, parts inventory, and airworthiness status is essential for safety and regulatory compliance. My goal was to recreate a small-scale but realistic version of such a system and use it as a way to practice building, containerizing, and deploying multiple services.

To do this, I developed a backend API, a frontend web interface, and a PostgreSQL database. All three components were containerized using Docker and orchestrated with Docker Compose to ensure consistent configuration, reliable communication between services, and an automated deployment workflow.

1. Architecture Overview

The system is made up of three independent services, each running in its own container and connected through Docker Compose networking.

Backend API

The backend exposes two main endpoints:

- `/status` — used for health checks
- `/items` — retrieves data stored in the database

All configuration values (database host, port, credentials, and API port) are handled through environment variables, which makes the service flexible across different environments.

The backend image is built using a multi-stage Dockerfile and runs as a non-root user to improve security.

Frontend

The frontend is a lightweight web interface that communicates with the backend API and displays the retrieved data.

It is also built using a multi-stage Dockerfile:

- The first stage compiles the frontend assets
- The second stage serves the static files using Nginx

This approach ensures that the final production image remains small and efficient.

Database

The database service uses the official `postgres:15` image.

Configuration is handled through environment variables, and a health check ensures the database is fully ready before the backend starts.

Service Interactions

The frontend communicates only with the backend API, and the backend handles all interactions with the database.

Docker Compose manages the internal network and startup order, ensuring services launch in the correct sequence.

2. Key Commands Used

These Docker and Docker Compose commands were central to my workflow:

Build container images:

```
docker compose build
```

Validate the Compose configuration:

```
docker compose config
```

Start the full stack in detached mode:

```
docker compose up -d
```

Tag images before pushing to Docker Hub:

```
docker tag aircraft_backend:latest charlotte254/aircraft_backend:latest  
docker tag aircraft_frontend:latest charlotte254/aircraft_frontend:latest
```

Push images to Docker Hub:

```
docker push charlotte254/aircraft_backend:latest  
docker push charlotte254/aircraft_frontend:latest
```

Clean up stopped containers:

```
docker container prune
```

3. Best Practices Followed

Multi-Stage Builds

Both the backend and frontend use multi-stage Dockerfiles, which significantly reduce the size of the final images:

- Frontend: **74.4 MB**
- Backend: **262 MB**

Without multi-stage builds, these images would be much larger and less efficient.

Non-Root Execution

All custom containers run as non-root users, which reduces the impact of potential vulnerabilities.

.dockerignore

I used a `.dockerignore` file to exclude unnecessary files such as `node_modules` and virtual environments from the build context. This improves build speed and keeps images clean.

Environment Variables

All sensitive information and environment-specific configuration values are stored in environment variables rather than hard-coded in the code or Dockerfiles.

Health Checks

Health checks were defined for each service to ensure that dependent containers only start when their prerequisites are ready.

Lightweight Base Images

Using lightweight base images such as `python: slim` and `nginx: alpine` helped reduce image size and minimize the attack surface.

4. Automation

I created a Bash deployment script (`deploy.sh`) to automate the entire deployment workflow.

The script performs the following steps:

1. Validates the Docker Compose configuration

2. Builds the container images
3. Logs in to Docker Hub
4. Tags the images
5. Pushes the images to Docker Hub
6. Stops running containers
7. Restarts the stack using `docker compose up -d`

This automation reduces manual effort, improves reliability, and minimizes the risk of human error during deployment.

5. Security Practices

Security was an important part of this project, especially because real aircraft maintenance systems deal with sensitive operational data. I applied several security-focused practices throughout the development process.

I made sure all containers run as non-root users and used lightweight base images to reduce the number of installed packages. Sensitive information is stored in environment variables, and `.dockerignore` helps prevent unnecessary files from being included in the build context.

I also limited exposed ports and relied on Docker's internal network for communication between services. Health checks were added to control service startup and prevent unstable containers from running.

To protect image integrity, I enabled Docker Content Trust to sign my images before pushing them to Docker Hub. This ensures that the images haven't been tampered with and that only trusted versions can be pulled.

Finally, I scanned my backend image using Snyk. The scan reported two types of vulnerabilities:

- **Base image vulnerabilities (Debian 13)** — all low-severity and dependent on upstream updates
- **Python dependency vulnerabilities** — mainly related to `flask-cors`, which can be fixed by upgrading the package

There were no critical or high-severity vulnerabilities, which means the image is generally safe to use.

6. Difficulties and Possible Improvements

Difficulties Encountered

During development, I ran into several challenges:

- Some slim images didn't include tools like `curl`, which caused health check failures at first
- Docker Compose sometimes created duplicate containers when the project name changed
- Environment variables weren't always available to the deployment script unless exported correctly
- Docker Hub authentication failed when the username or password variables were empty
- Old containers accumulated over time and had to be cleaned up manually

Possible Improvements

Future improvements could include:

- Implementing a CI/CD pipeline
- Adding database migration support
- Introducing a reverse proxy such as Traefik
- Enabling HTTPS
- Scaling services or migrating to Kubernetes
- Applying more advanced container security configurations

Conclusion

This project gave me practical experience in building and deploying a containerized web application using Docker. I strengthened my understanding of backend, frontend, and database containerization, service orchestration with Docker Compose, and deployment automation.

The final solution is clean, reproducible, and aligned with modern best practices. It also reflects the structure and workflows commonly used in a real-world production environment.